

EXTRACTED SCOPE DOCUMENT

AI Automation & Copilot

What AI-related work you actually need to build inside the Dental Lead Management CRM project — capabilities, guardrails, data model, screens, and a practical build order.

Source document	Dental CRM + AI Platform Specification (10 July 2026)
Focus of this PDF	Section 05C (AI Automation & Copilot) + Section 11 (Automation examples)
AI stack mandated	Vercel AI SDK / AI Gateway, Zod structured outputs, Supabase pgvector
Core rule	Human-in-the-loop — AI recommends and drafts, humans approve and send

Table of Contents

- 1. What AI Automation Means in This Project** **3**
 - Golden rule3
 - Two different things you'll build3
- 2. The 7 AI Capabilities You Need to Build** **4**
- 3. AI Governance — Non-Negotiable Rules** **6**
 - Why this matters for you as the builder6
- 4. Technology Stack for the AI Layer** **7**
 - AI-specific database tables you'll need7
 - Screens/routes to build7
- 5. Rule-Based Automation Examples** **8**
 - AI Operations dashboard8
- 6. What NOT to Build (Explicit Exclusions)** **9**
 - Risk this protects against9
- 7. Practical Build Order (Recommended)** **10**
- 8. Quick-Reference Checklist** **11**

1. What AI Automation Means in This Project

The specification treats AI as an **operational assistant layer** sitting on top of the CRM — not a separate product. Every AI feature reads real CRM data (leads, calls, appointments) and produces a suggestion, summary, score, or draft. Nothing AI produces is allowed to silently change data, send a message, or make a decision on its own — a human always reviews and approves the consequential actions.

Golden rule

Golden Rule — AI may recommend and draft. Only a human approves changes, communications, and any patient-facing action. This single rule shapes almost every AI feature you'll build.

Two different things you'll build

Layer	What it is	Example
AI capabilities (Section 05C)	Model-powered features: classification, summarization, drafting, Q&A	Summarize a call transcript and suggest next action
Rule-based automation (Section 11)	Deterministic if-this-then-that logic — no model call needed	Auto-create a follow-up task when call outcome = "No Answer"

Both live under the same "automation" umbrella in the product, but they're built differently — rule-based automation is plain application logic; AI capabilities call a language model through the Vercel AI SDK and must go through the governance rules in Section 3 of this document.

2. The 7 AI Capabilities You Need to Build

This is the direct checklist from Section 05C of the specification — each row is a distinct feature with its own trigger and output.

1. Lead Triage

TRIGGER	New or updated lead
OUTPUT	Classify treatment interest and source, run a completeness check, and produce an explainable priority score (not just a number — a reason for the score).

2. Conversation Intelligence

TRIGGER	Call transcript or message thread
OUTPUT	Summary of the conversation, detected intent, objections raised, sentiment, any commitments made, and a recommended follow-up.

3. Agent Copilot

TRIGGER	User opens a lead
OUTPUT	Next-best-action suggestion, a proposed task, a draft reply, and relevant history — each claim linked back to its source record.

4. Appointment Intelligence

TRIGGER	Booking and reminder events
OUTPUT	No-show risk flag, a recommended reminder channel/time, and a recovery workflow for at-risk appointments.

5. Management Intelligence

TRIGGER	Scheduled or threshold event
OUTPUT	Narrative KPI summary in plain language, anomaly detection, backlog risk, and suggested links to investigate further.

6. Knowledge Assistant

TRIGGER	Natural-language question from a user
OUTPUT	A permission-filtered answer with tool results — never unrestricted SQL, never cross-clinic data the user isn't allowed to see.

7. Data Quality

TRIGGER

Import or record update

OUTPUT

Duplicate-lead candidates, inconsistent status/date warnings, and normalization suggestions (e.g. phone number formatting).

3. AI Governance — Non-Negotiable Rules

These aren't feature requests — they're constraints every AI feature above must satisfy before it can ship.

- Provider-neutral: build through the Vercel AI SDK (+ optional AI Gateway) so the underlying model can be swapped without rewriting business logic.
- Every automation that writes a classification, task, or recommendation must use structured output validated with Zod/JSON schema — never free-form text written straight to the database.
- No autonomous clinical diagnosis, treatment recommendation, emergency triage, or medical decision-making, ever.
- Outbound messages, lead closure, material status changes, payments, and any destructive action require explicit user approval before they happen.
- Log everything: prompt, model, temperature/options, retrieved sources, token usage, latency, cost, output, and user feedback — for every AI run.
- Apply PII minimization / redaction before sending data to a third-party model, wherever the workflow allows it.
- Define an offline evaluation set, acceptance thresholds, fallback behavior, and a kill switch before any AI feature goes to production.
- Long-running work (transcription, embeddings, batch analysis) runs in queued background jobs with retries and idempotency — never blocking the interactive UI.

Why this matters for you as the builder

In practice this means every AI feature needs three things beyond the model call itself: (1) a structured schema so the output is safe to store, (2) an approval step in the UI before anything user-facing happens, and (3) a logging table entry so the run is auditable later. Build these three as shared infrastructure once, then reuse them for all 7 capabilities instead of rebuilding per-feature.

4. Technology Stack for the AI Layer

Piece	Required choice	What it's for
AI application layer	Vercel AI SDK	Streaming UI, structured outputs, provider abstraction, tool calling, copilot UX
Model routing	Vercel AI Gateway or an approved provider	Budget controls, usage visibility, provider fallback, per-workflow model choice
Vector search	Supabase pgvector	Permission-aware semantic search over approved summaries/knowledge documents
Validation	Zod schemas	Forces every AI output into a safe, typed shape before it touches the database
Background jobs	Supabase Cron/Edge Functions, or Trigger.dev/QStash for heavier work	Runs transcription, embeddings, and batch AI analysis without blocking the UI

AI-specific database tables you'll need

From the data model section — these are the tables that make AI auditable and controllable, not just "smart":

Table	Purpose
automation_rules	Definitions of rule-based automations (Section 11 examples)
automation_runs	Execution log for rule-based automation triggers
ai_runs	Every AI model call — inputs, outputs, cost, latency, status
ai_prompt_versions	Versioned prompts so you can roll back/compare changes
ai_feedback	User accept/edit/reject feedback on AI suggestions
ai_usage	Token/cost tracking per user, org, and feature
knowledge_documents / knowledge_chunks / embeddings	Source content and vector embeddings for the Knowledge Assistant's semantic search

Screens/routes to build

- /ai/copilot — the in-context assistant panel (Agent Copilot capability)
- /ai/runs — a review log where staff can see and give feedback on past AI runs
- /automations — where managers configure rule-based automation logic

5. Rule-Based Automation Examples

These don't need a language model at all — they're plain conditional logic, but they live in the same "automation" system and should share the same audit-log pattern as the AI features.

- Assign new leads using round-robin or clinic/treatment rules.
- Create a follow-up task automatically when a call outcome is "No Answer" or "Callback Requested".
- Send an appointment confirmation right after booking, and a reminder before the appointment.
- Escalate overdue high-priority leads to a clinic manager.
- Notify managers about unassigned leads or unusual no-show rates.
- Generate scheduled management reports — only after Phase 1 reporting numbers are reconciled.
- Summarize calls and propose tasks — without auto-sending anything or silently changing lead status.
- Prioritize newly received leads using transparent, manager-adjustable factors.
- Draft personalized follow-ups from approved templates plus CRM context, for agent review.
- Create anomaly alerts for sudden drops in contact rate, attendance, or clinic conversion.

AI Operations dashboard

The spec also requires a dedicated dashboard just for monitoring the AI layer itself — treat this as part of the AI automation scope, not an afterthought:

Widget	Shows
AI operations dashboard	Runs, success/failure rate, acceptance rate, edit rate, latency, tokens/cost, model used, and automation outcomes

6. What NOT to Build (Explicit Exclusions)

Equally important as the checklist — these are explicitly out of scope unless the client separately adds them:

- Autonomous clinical diagnosis, treatment recommendations, emergency triage, or unreviewed medical communication of any kind.
- AI sending a message, closing a lead, or changing a material status without a human clicking "approve" first.
- Unrestricted SQL access or any tool that could return cross-clinic / cross-organization data to the Knowledge Assistant.
- Electronic medical records, diagnosis, prescriptions, or clinical charting — this system stays a CRM, not a clinical record system.

Risk this protects against

Risk	Mitigation you should implement
AI hallucination or unsafe action	Restrict use cases, structured output, source links, human approval, evaluations, kill switch
AI privacy and cost overrun	PII policy, server-only API keys, provider review, budgets, model routing, usage alerts

7. Practical Build Order (Recommended)

The specification doesn't mandate a sequence, but based on dependencies, this order minimizes rework:

Step 1

Build the shared AI infrastructure first

ai_runs / ai_prompt_versions / ai_feedback / ai_usage tables, the Vercel AI SDK wrapper, Zod schema validation helper, and an approval-UI pattern you'll reuse everywhere.

Step 2

Data Quality + Lead Triage

Simplest capabilities — classification and scoring on structured lead data you already have; no transcripts or long text needed yet.

Step 3

Conversation Intelligence

Requires call/message data to exist first (Section 5.5) — summarization, intent, sentiment.

Step 4

Agent Copilot

Builds on Steps 2–3: next-best-action pulls from triage scores and conversation summaries.

Step 5

Appointment Intelligence

Needs appointment history to exist and enough volume for no-show risk to be meaningful.

Step 6

Knowledge Assistant (pgvector)

Set up embeddings + tenant-aware retrieval functions; this is the most security-sensitive capability, build it after your RLS patterns are solid.

Step 7

Management Intelligence + AI Ops Dashboard

Narrative summaries and anomaly detection depend on reporting/KPI definitions (Section 12) being reconciled first.

8. Quick-Reference Checklist

Use this as your literal task list when scoping the AI automation work in your project plan.

- Vercel AI SDK integrated with a provider (via AI Gateway or direct)
- Shared `ai_runs` / `ai_prompt_versions` / `ai_feedback` / `ai_usage` tables + logging wrapper
- Zod schema validation on every AI output before it's stored
- Approval UI pattern for any AI output that leads to a send/status-change/destructive action
- Lead Triage: classification + explainable priority score
- Conversation Intelligence: summary, intent, sentiment, commitments, follow-up suggestion
- Agent Copilot: next-best-action, task proposal, reply draft, source-linked history
- Appointment Intelligence: no-show risk flag + reminder optimization + recovery workflow
- Management Intelligence: narrative KPI summaries + anomaly detection
- Knowledge Assistant: permission-filtered Q&A over pgvector, tenant-scoped retrieval
- Data Quality: duplicate detection + normalization suggestions on import/update
- Rule-based automations (10 examples in Section 5) wired to `automation_rules/automation_runs`
- AI Operations dashboard (runs, acceptance rate, latency, cost, model used)
- PII minimization/redaction before third-party model calls
- Evaluation set, acceptance thresholds, fallback behavior, and kill switch defined pre-launch

Note: This document summarizes only the AI Automation & Copilot portion of the full platform specification. The complete document also covers the public website, CRM modules, database design, security, migration, DevOps, and delivery phases — refer back to the full specification for those areas.